



Maastricht University

Project 2.2 Report

Can Synthetic Data Replace Real Data for Pedestrian Trajectory
Prediction?

DATA SCIENCE & AI

Authors:

Ciprian Driscu, Claire Bams,
Keez Cuijpers, Néo Deward,
Nick Grebe and Noah Nuelandt

August 21, 2026

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Research Questions	3
2	Methods	3
2.1	Data	3
2.2	Approach	3
2.2.1	Feature Extraction	3
2.2.2	Social-LSTM	4
2.2.3	Spatio-Temporal Transformer	5
2.3	Experimental Setup	10
3	Results	11
3.1	Statistical Analysis	11
3.2	Quantitative Results	14
3.2.1	RQ1: Spatio-Temporal Transformer and LSTM	14
3.2.2	RQ2: Real vs. Synthetic Training	14
3.2.3	RQ3: Cross-Domain Generalisation	15
4	Discussion	16
4.1	Interpretation of Findings	16
4.2	Limitations	17
4.3	Future Work	17
5	Conclusion	17
A	Supplementary Visualisations and Performance Metrics	18

Abstract

Main writer: Keez Cuijpers

Reviewer: Drişcu Ciprian

Collecting and annotating real-world data for pedestrian trajectory prediction is expensive, privacy-sensitive, and difficult to scale. Synthetic data generated from game engines offers a privacy-safe alternative, but whether it can match the predictive quality of real-world training data remains an open question. This study directly compares models trained on real and synthetic data using an identical evaluation pipeline across three research questions: model architecture performance, real-versus-synthetic training quality, and cross-domain generalization.

A custom Spatio-Temporal Transformer (STT) and a Social Long Short-Term Memory (Social-LSTM) baseline are trained and evaluated on MOT20 (real-world) and MOTSynth (GTA V-based synthetic), reporting Average Displacement Error (ADE) and Final Displacement Error (FDE) at horizons of 0.6s, 1.0s, and 2.0s.

The STT outperforms the Social-LSTM by 36–38% across all horizons $p = 6.0 \times 10^{-4}$ after holm adjustment. Across 20 synthetic data fractions ranging from 5% to 100% of MOTSynth, the synthetic-trained model consistently remained outside the $\pm 5\%$ equivalence margin relative to the real-trained model, with the performance gap failing to close as synthetic volume increased. Cross-domain evaluation revealed a strong asymmetry: the real-trained model generalised to both domains, while the synthetic-trained model performed well only within its own domain. These findings suggest that in the current setup, synthetic data alone cannot substitute for real-world training data for pedestrian trajectory prediction.

1 Introduction

1.1 Problem Statement

Main writer: Claire Bams

Reviewer: Drişcu Ciprian

Predicting pedestrian trajectory is an essential task in computer vision. This is useful in crowd monitoring, public safety, and self-navigating robotic systems. By being able to accurately predict the future trajectory of a pedestrian, the system can anticipate human movement, prevent collisions, and better manage crowded spaces. However, it remains challenging to make such predictions due to interaction between other people, varying movement patterns and complex environments.

The main problem in building this type of model is the requirement for large amounts of annotated real-world data. Furthermore, collecting and annotating real-world data is expensive and raises privacy concerns under regulations such as GDPR (European Data Protection Board, 2020). The need for better data solutions is clear when looking at the risks faced by pedestrians in traffic. Pedestrians constitute 17.8% of all fatalities in road accidents across the EU in 2024, self-driving cars require up to 5 seconds to react safely and the number of people in cities is expected to greatly increase by 2050, which further raises the demand for scalable data collection solutions (Eurostat, 2026; Wilson et al., 2023).

Synthetic data is a promising alternative. It provides perfect labelling, scalable scenario diversity, and is more likely to be privacy-safe. Prior work like STAR (Yu et al., 2020) achieves robust trajectory predictions, but it was trained and evaluated strictly on annotated real-world datasets. SimAug (Liang et al., 2020) showed that models trained on synthetic data can work

with real-world videos but they never actually compared the two types of data using the same pipeline. This project fills that gap by directly measuring the performance difference between models trained on real data, synthetic data.

1.2 Research Questions

Main writer: Claire Bams

Reviewer: Nick Grebe, Keez Cuijpers, Drişcu Ciprian

1. **Transformer comparison:** Does a crowd-aware Spatio-Temporal Transformer (STT) predict pedestrian trajectories more accurately than a recurrent Social-LSTM baseline of comparable scale, measured by Average Displacement Error (ADE) and Final Displacement Error (FDE) at prediction horizons of 0.5s (and if necessary an approximation of 0.6s), 1.0s, and 2.0s on the MOTChallenge benchmark?
2. **Synthetic vs. real training:** Can a Spatio-Temporal Transformer trained on a large-scale synthetic dataset achieve comparable pedestrian trajectory prediction accuracy, measured by ADE and FDE, to the same model trained on a smaller real-world dataset?
3. **Cross-domain generalization:** How does the cross-domain generalisation performance of a Spatio-Temporal Transformer differ when trained on real-world versus synthetic data, as measured by ADE and FDE across both real-world and synthetic test sets?

2 Methods

2.1 Data

Main writer: Claire Bams

Reviewer: Keez Cuijpers, Drişcu Ciprian

Two datasets belonging to the MOTChallenge benchmark family are used: MOT20 and MOTSynth. These two datasets were chosen because they share the same annotation format, which makes comparison between real and synthetic simple.

The MOT20 (Dendorfer et al., 2021) dataset is a real-world dataset which focuses on tracking pedestrians moving in physical environments. The footage is captured by a fixed overhead camera. Each pedestrian is annotated with a bounding box and a tracking identity for every frame. The main challenge with this dataset is the high crowd density, with some sequences having up to 200 pedestrians at the same time. This makes it more challenging to work with the MOT20. The total size is approximately 5 GB.

The MOTSynth (Fabbri et al., 2021) dataset is a collection of hundreds of video sequences generated using the GTAV game engine. The dataset is much larger, about 80 GB. It offers a large volume of data for training, perfect labelling, diverse scenarios, and no privacy concerns.

2.2 Approach

2.2.1 Feature Extraction

Main writer: Claire Bams

Reviewer: Nick Grebe

Nine features are extracted from each pedestrian’s bounding box. The place where the pedestrian’s feet touch the ground is taken as being the bottom center of their bounding box, computed as the horizontal center and the bottom edge. After that, the coordinates are normalised using the dimension of the largest image, with a small centering adjustment applied to account for non-square images:

$$x_{norm} = \frac{x}{d_{max}} + \frac{1 - w/d_{max}}{2}, \quad y_{norm} = \frac{y}{d_{max}} + \frac{1 - h/d_{max}}{2}$$

where $d_{max} = \max(w, h)$, w is the image width, and h is the image height, ensuring the features are independent of camera resolution and viewpoint, guaranteeing consistency across MOT20 and MOTSynth.

The following 9 features are computed for each pedestrian per frame:

- Normalised foot position (x, y)
- Frame-to-frame displacement (dx, dy)
- Velocity (vx, vy), computed as displacement divided by the time between frames
- Speed magnitude, computed as the Euclidean distance divided by the time between frames
- Movement direction ($\sin \theta, \cos \theta$), encoding the heading as a unit vector

All these features were chosen because they help understand the full kinematic state of a pedestrian: where they are, how fast they are going, and in which direction. Direction is encoded as $\sin \theta$ and $\cos \theta$ rather than a raw angle to avoid the discontinuity at 0 and 360 degrees. For the first observed frame of a track, there is no prior position to compare against, so the displacement is set to zero.

The observation window looks up to $T = 15$ time steps in the past, with a stride of 5 frames, meaning every fifth frame is sampled. The time between observations is therefore computed as stride/framerate, where the frame rate is obtained directly from the metadata of each sequence. When a pedestrian is not visible in a certain frame, the corresponding feature vector is set to zero and a binary mask records which frames are valid. The result is a fixed-length feature sequence per pedestrian covering the observed past window, which is then passed as input to the model.

2.2.2 Social-LSTM

Main writer: Keez Cuijpers

Reviewer: Drişcu Ciprian

The Social-LSTM is a recurrent baseline built on the architecture introduced by Alahi et al., 2016. It processes each pedestrian’s position history sequentially through a single-layer LSTM encoder and generates future positions autoregressively through a single-layer LSTM decoder. In contrast to the Spatio-Temporal Transformer, which takes the full nine-dimensional kinematic feature vector described in Section 2.2.1 as input, the Social-LSTM operates only on normalized two-dimensional foot positions. This simpler input is consistent with the original formulation of Alahi et al., 2016. As both the input representation and the training objective differ from those of the Spatio-Temporal Transformer, the comparison between the two models

reflects a difference in modelling approach rather than a like-for-like architectural comparison. The Social-LSTM was selected because it shares the same motivation of modelling crowd interactions, while using a recurrent rather than attention-based architecture, making it a meaningful point of comparison.

Each two-dimensional position observation is first projected to a 64-dimensional embedding through a linear layer before being passed into the encoder LSTM with a hidden size of 128. The encoder processes the full observation window of 15 timesteps step by step, updating a fixed-size hidden state at each frame. Because information must be compressed into this hidden state, earlier observations have progressively less influence on the final representation as the sequence grows longer. This temporal decay is a known limitation of recurrent architectures and is one of the primary motivations for the transformer-based approach described in the following section.

Social interaction is modelled through an occupancy-based pooling mechanism applied at each observed timestep. A 4×4 spatial grid is placed around each pedestrian, covering a neighbourhood radius of ± 0.5 normalized units in both spatial dimensions. The hidden states of all pedestrians whose relative position falls within this window are max-pooled into their corresponding grid cell. The resulting $4 \times 4 \times 128$ grid is flattened to a 2048-dimensional vector and linearly projected back to the 128-dimensional hidden space, after which it is added to the encoder hidden state. This gives each pedestrian a socially-aware representation of their local crowd context at every timestep, following the social pooling design of Alahi et al., 2016. Max-pooling per cell is used rather than summing, to emphasize the most influential nearby agent within each spatial region rather than accumulating contributions uniformly.

Future positions are produced autoregressively by a decoder LSTM of the same hidden size. The decoder is seeded with the embedding of the last observed absolute position and at each step outputs a two-dimensional displacement $(\Delta\hat{x}, \Delta\hat{y})$. Absolute future positions are recovered by cumulatively summing displacements from the last observed foot position. Ten future steps are predicted in total, matching the Spatio-Temporal Transformer’s prediction horizon to ensure a direct comparison.

The model is trained using a masked mean squared error over predicted displacements:

$$\mathcal{L}_{\text{LSTM}} = \frac{1}{V} \sum_{t,n} m_{t,n} \cdot \|\Delta\hat{p}_{t,n} - \Delta p_{t,n}\|^2 \quad (1)$$

where $m_{t,n} \in \{0, 1\}$ is the future validity mask and $V = \sum_{t,n} m_{t,n}$ is the total number of valid future positions. This is a simpler training objective than the combined SmoothL1 loss used for the Spatio-Temporal Transformer. The optimizer is Adam with a learning rate of 1×10^{-3} , no weight decay, and no learning rate scheduler, trained for a maximum of 50 epochs with a batch size of 32. Training stops early if the loss improvement between epochs falls below 10%.

2.2.3 Spatio-Temporal Transformer

Main writer: Claire Bams

Reviewer: Nick Grebe

The trajectory prediction model is a custom Spatio-Temporal Transformer. It processes history of all pedestrians simultaneously and predicts their future positions. A transformer-based architecture was chosen over recurrent models such as the Social-LSTM because self-attention can attend to any point in the observed past equally regardless of how far back it occurred.

Other models, like LSTMs have a problem where they tend to forget earlier information as the sequence gets longer, which limits their ability to capture long-term temporal relationships. Furthermore, the transformer model is also faster and more stable because it processes all timesteps in parallel rather than sequentially. The spatial component was added on top of the temporal transformer to model crowd interactions, since pedestrian trajectories are influenced by surrounding individuals. Without the spatial component, it would miss these interactions. The model is built around seven sequential blocks and 477,315 trainable parameters.

Motion Feature Stem

Main writer: Nick Grebe

Reviewer: Keez Cuijpers

The motion feature stem is an embedding block that converts each valid 9-dimensional motion observation into a 128-dimensional hidden representation. The invalid observations, which mean a specific time step has no recorded position, are identified using the temporal validity mask. The temporal validity mask zeros out the invalid time steps before the first linear layer so missing or padded observation cannot meaningfully contribute motion information to the model. The mapping is learnt by a $9 \rightarrow 64$ linear layer using GELU, which was chosen as a smooth alternative to the RELU function (Hendrycks & Gimpel, 2016). This is followed by another linear layer with $64 \rightarrow 128$, which is then Layer normalized and finalized by $p = 0.05$ Dropout regularization to combat overfitting (Ba et al., 2016; Srivastava et al., 2014).

This whole block serves as the foundation for the later attention layers, which operate best on a shared hidden representational space, instead of on raw coordinates, velocities and headings. The use of positional, heading, velocity and speed parameters can be easily justified by classical and learning based trajectory forecasting (Salzmann et al., 2020; Schöller et al., 2020). The specific two-linear-layer stem and hidden size were implementational decisions for a manageable model size for the project’s time frame.

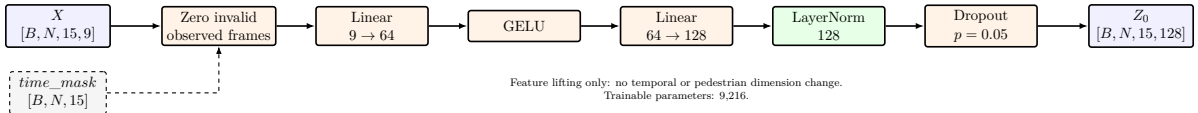


Figure 1: Motion Feature Stem Block

Spatial Interaction Block

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

The spatial interaction block makes each pedestrian embedding aware of nearby individuals at the same timestep. Entering this block each pedestrian has a specific motion embedding of the form, $Z_0 \in \mathbb{R}^{B \times N \times 15 \times 128}$, lacking the information about how each pedestrian relates to k other pedestrians. This block addresses that gap.

This is achieved by building a masked k -nearest-neighbor graph using each pedestrian’s current kinematic state, computing edge features for nearby pedestrians and scoring the edges with a small MLP. The MLP consists of two linear layers from $8 \rightarrow 64$ and $64 \rightarrow 128$ with GELU as the activation function, followed by a linear layer outputting an edge score (Huang et al., 2019; Yu et al., 2020). This results in the model learning to place higher importance on

close, influential pedestrians, and lower importance on the non-influential ones. The resulting embedding is therefore kinematically aware and socially rich.

This block is justified through the hypothesis that paths in a crowd are not independent. People slow down when obstructed by others, may even change direction, or follow paths based on nearby agents. This design is motivated by social pooling in Social-LSTMs, graph-based spatial modelling in Social-STGCNN, attention-based social interaction modelling in STGAT and most of all by the spatio-temporal graph idea in the STAR architecture (Alahi et al., 2016; Huang et al., 2019; Mohamed et al., 2020; Yu et al., 2020).

The choice of $k = 3$ neighbors keeps the graph local and importantly computationally light, while still allowing each pedestrian to receive information from relevant nearby pedestrians, representing a tradeoff between social context and computational cost.

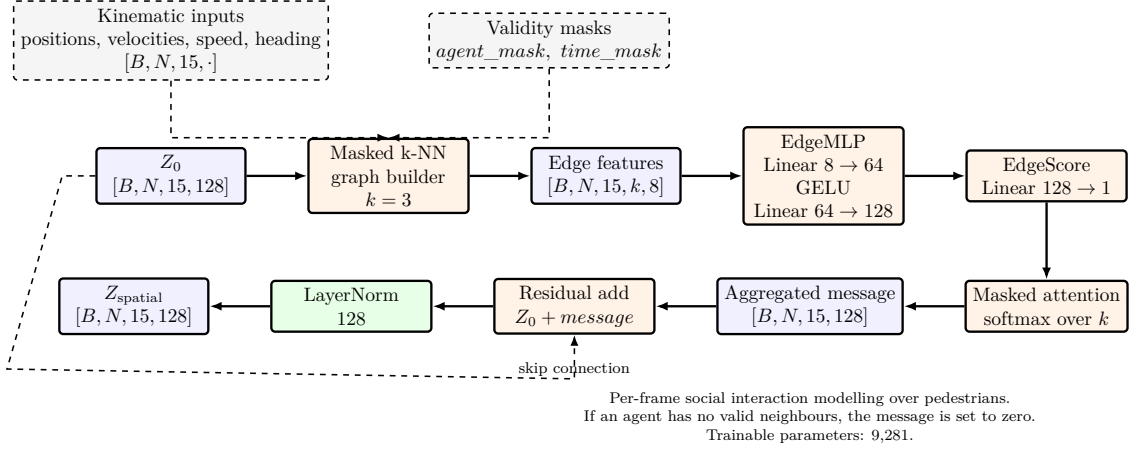


Figure 2: Spatial Interaction Block.

Temporal Patching

Main writer: Nick Grebe

The temporal patching block compresses the 15 observed timesteps into 5 temporal tokens, each of which summarizes a brief 3-frame motion segment. The input $Z_{spatial} \in \mathbb{R}^{B \times N \times 15 \times 128}$ tensor is reshaped to $[B \cdot N, 128, 15]$, on which then a 1D convolution is applied, followed by a GELU and LayerNorm. With $kernelsize = 3, stride = 3$ every patch covers 3 observed frames. Thus the output becomes $H \in \mathbb{R}^{B \times N \times 5 \times 128}$, making the attention mechanism cheaper.

This follows the general transformer motivation of representing sequences as tokens that attend to each other and is found in similar approaches that reason in a similar way (Yu et al., 2020; Yuan et al., 2021). The use of temporal convolution is motivated by the observation that local movement patterns are directly influential and informative of future movement. This can also be seen by the strong constant-velocity baseline (Schöller et al., 2020).

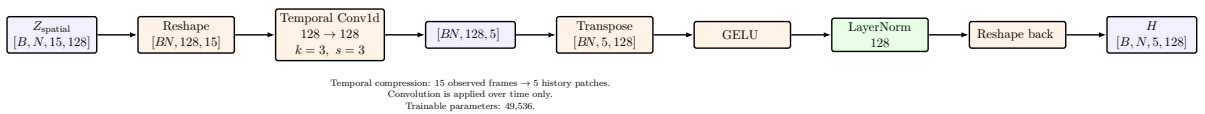


Figure 3: Temporal Patching Block

Crowd Context Fusion

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

The crowd context fusion block gives every pedestrian a lightweight summary of the overall crowd movement. It receives the result of the temporal patching and then pools over the pedestrian dimension N , computing $C = \text{maskedmean}(H)$. It produces one crowd-summary token per temporal patch: $C \in \mathbb{R}^{B \times 1 \times 5 \times 128}$. This crowd-summary token is then copied to the pedestrian dimension. Every pedestrian has the tensor shape of $[B, N, 5, 128]$, thus C has to be adjusted to $[B, N, 5, 128]$ as well by giving every pedestrian the same crowd level context: The crowd context is broadcast across the pedestrian dimension:

$$C_{\text{broadcast}}[b, n, k, :] = C[b, 0, k, :]$$

Finally, the fusion is computed as:

$$H_{\text{crowd}}[b, n, k, :] = \text{LayerNorm}(H[b, n, k, :] + C_{\text{broadcast}}[b, n, k, :])$$

This block is necessary as up to now, in the model has built a purely local neighborhood graph that may miss global crowd conditions, especially prevalent in modern infrastructure such as shopping malls, pedestrian walkways and underpasses. This design decision is motivated by previous social trajectory forecasting methods that treat the surrounding crowd as useful context, as in the other previously mentioned models (Alahi et al., 2016; Huang et al., 2019; Mohamed et al., 2020). In this implementation the context is purposefully lightweight, using only pooling LayerNorm parameters while still giving each pedestrian access to a global scene descriptor.

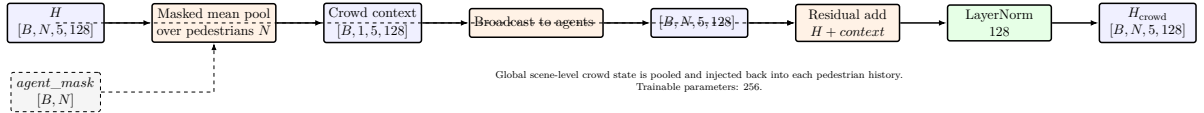


Figure 4: Crowd Context Fusion Block.

Future Token Block

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

The future token block prepares empty but learnable "future slots" that can be attended to by the encoder history and become prediction specific representations. It does this by appending learned positional embeddings to the 5 historical tokens represented as such: $H_{\text{crowd}} \in \mathbb{R}^{B \times N \times 5 \times 128}$. The model must output 10 future positions hence it needs 10 future "slots" to be able to predict into. The block therefore uses learnable future tokens. $F_{\text{base}} \in \mathbb{R}^{1 \times 1 \times 10 \times 128}$, which is broadcasted to all possible and valid pedestrians $F \in \mathbb{R}^{B \times N \times 10 \times 128}$. Followed by the addition of the five historical tokens $P_{\text{hist}} \in \mathbb{R}^{1 \times 1 \times 5 \times 128}$ and the ten future tokens $P_{\text{fut}} \in \mathbb{R}^{1 \times 1 \times 10 \times 128}$, giving $Z_{\text{in}} = [H_{\text{crowd}} + P_{\text{hist}} || F + P_{\text{fut}}]$ resulting in $Z_{\text{in}} \in \mathbb{R}^{B \times N \times 15 \times 128}$. This design is inspired by encoder-style forecasting in transformer approaches where learned tokens are used to represent different agents and time steps (Yu et al., 2020; Yuan et al., 2021).

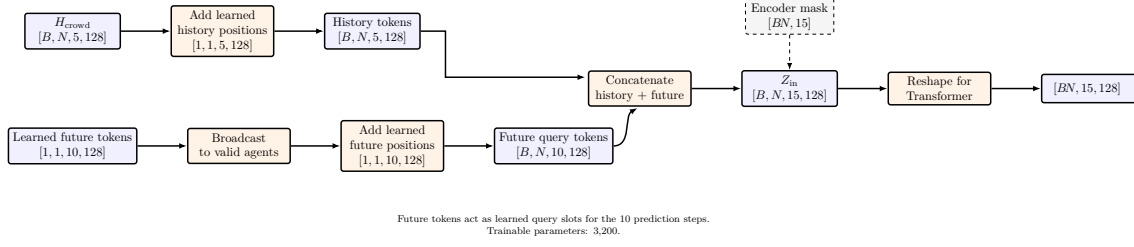


Figure 5: Future Token Block

Transformer Encoder

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

The transformer encoder models temporal dependencies within each pedestrian sequence, after the cross pedestrian interaction has already been built by previous stages, turning the empty future query tokens into representation of future oriented ("oriented" as the actual future positions are only produced in the later regression head) positions by enabling attention of previously seen kinematic states. This is done by combining the encoded history tokens and the learnt future query tokens. The input to the encoder is $Z_{in} \in \mathbb{R}^{B \times N \times 15 \times 128}$, which is reshaped into the $[B \cdot N, 15, 128]$ tensor, thus each pedestrian becomes a singular transformer sequence. As the pedestrians themselves have already been injected by earlier blocks, this self attention mechanism allows future query tokens to attend to the encoded history tokens and also to each other, producing representations that model the future based on the observed trajectory.

The architecture consists of three transformer encoder layers with a hidden size of 128, four attention heads, a feedforward dimension of 256, GELU activation, dropout of $p = 0.1$ for gentle regularisation, and prenormalisation. This is specifically motivated by the STAR and AgentFormer models (Yu et al., 2020; Yuan et al., 2021).

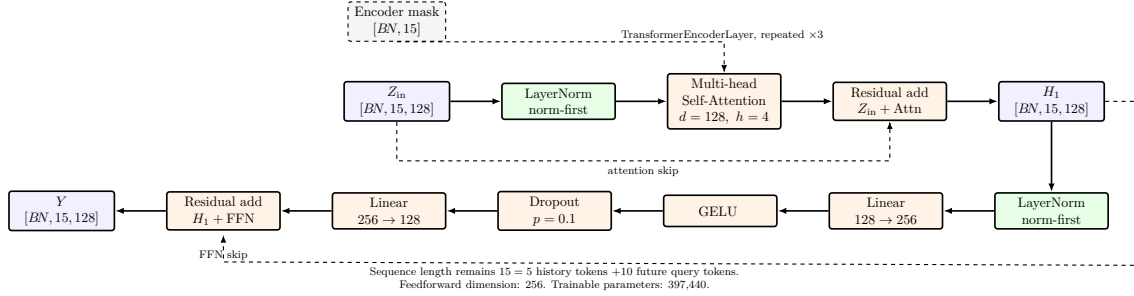


Figure 6: Transformer Encoder Block

Regression Head and coordinate reconstruction

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

The regression head is the final block and converts the encoded future tokens into a concrete trajectory prediction. It receives the $Y \in \mathbb{R}^{B \times N \times 15 \times 128}$ shaped output from the transformer encoder of which then only the last 10 tokens are used $Y_{future} = Y[:, :, 5 :, :]$ giving $Y_{future} \in \mathbb{R}^{B \times N \times 10 \times 128}$. Then the regression head maps each 128-dimensional future token to a 2-dimensional displacement vector through the following sequence: $\text{Linear}(128 \rightarrow 64) \rightarrow$

$GELU \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(64 \rightarrow 2)$ such that the output is $[\Delta\hat{x}, \Delta\hat{y}]$, which is future movement. Finally this gets reconstructed as $\hat{P}_{future} = P_i + \text{cumsum}(\Delta\hat{P}, \text{dim} = 2)$. Predicting displacements is justified by earlier motion predicting work, showing the importance of local velocity and constant motion structures in pedestrian path forecasting (Alahi et al., 2016; Salzmann et al., 2020). During training, the model is optimised using a masked dual SmoothL1 defined as follows:

$$L = 0.5 \cdot L_{\Delta} + 1.0 \cdot L_{\text{position}}, \quad (2)$$

$$L_{\Delta} = \text{masked SmoothL1}(\Delta\hat{P}, \Delta P_{\text{true}}), \quad (3)$$

$$L_{\text{position}} = \text{masked SmoothL1}(\hat{P}_{\text{future}}, P_{\text{future, true}}). \quad (4)$$

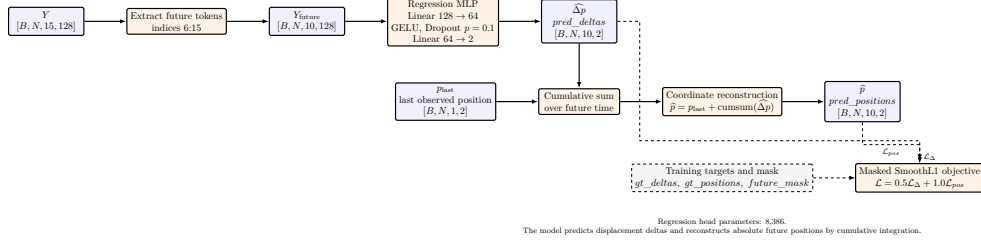


Figure 7: Regression Head and coordinate reconstruction.

2.3 Experimental Setup

Main writer: Claire Bams

Reviewer: Noah Nuelandt

The experimental setup is based upon the three research questions defined in the introduction. Each experiment helps to answer one of these questions. The average displacement error (ADE) and the final displacement error (FDE) are used to have a fair metric to compare the model performance. The evaluation is performed at three future time points, 0.6s, 1.0s, and 2.0s from the current time. Since the trajectory stride is fixed at 5 frames, and the MOT20 dataset has a frame rate of 25 FPS, the predicted time points may not match the evaluated points exactly. Predictions are rounded to the closest future timestep. For example, 0.5s is rounded to the closest frame, 0.6s. Each experiment varies only in one variable while keeping the other parameters the same. All models are evaluated on data that was not seen during training.

Experiment 1: Transformer comparison

Main writer: Claire Bams

Reviewer: Keez Cuijpers

Contributors: Noah Nuelandt

The objective of the first experiment is to determine whether the Spatio-Temporal Transformer outperforms a simpler baseline model. Two models are compared: the Spatio-Temporal Transformer against a Social-LSTM.

Both models use an observation window of 15 frames and predict 10 future steps, consistent with the Spatio-Temporal Transformer and with an almost identical number of trainable parameters (461,378 vs. 477,315).

We decided to make use of Leave-One-Out training (LOOCV) as we decided to split our training set by sequence instead of per person or per frame. The advantage with this approach is the ability to train on the entirety of our training set, which only consists of 4 separate sequences. This also generates 4 individual checkpoints, one for each sequence we place in the testing set.

Experiment 2: Synthetic vs Real Training

Main writer: Claire Bams

Reviewer: Noah Nuelandt

Contributors: Nick Grebe, Noah Nuelandt

In the second experiment, the objective is to determine whether it is possible to get similar accuracy with a model trained on synthetic data as one trained on real-world data, and the minimum synthetic training fraction required to achieve practical equivalence with the real-world-trained model. To perform this experiment, we have created an 80/10/10 split of MOTSynth, and ran training with a growing subset at a 5% rate per run. For each of the 20 generated checkpoints, the ADE and FDE are compared to the real-world baseline models generated during the first research question.

Experiment 3: Cross-Domain Generalisation

Main writer: Claire Bams

Contributors: Nick Grebe

Finally, the third experiment seeks to determine whether models perform better when trained and tested on the same type of data, or whether they can generalise across real and synthetic domains.

A set of spatio temporal transformer models are trained: LOO of the real world data from MOT20 with four held out models and one on trained on the entire MOTSynth synthetic dataset. Both model categories (real-world-trained and synthetic-trained) are tested on both real and synthetic held-out test sets, resulting in four conditions: real-to-real, real-to-synthetic, synthetic-to-real and synthetic-to-synthetic. The performance is evaluated using ADE and FDE for these four conditions.

3 Results

3.1 Statistical Analysis

Main writer: Nick Grebe

Reviewer: Keez Cuijpers

Evaluation units and metrics

The analysis is based on the exported evaluation units instead of only final summary scores. Every single unit stores the following: errors for a specific model, sequence, pedestrian track, prediction horizon and metric. The metrics for accuracy are ADE and FDE, both commonly used in trajectory prediction evaluation (Mohamed et al., 2022; Uhlemann et al., 2024). ADE measures the average Euclidean distance between predicted and ground truth positions over the prediction horizon. FDE measures the Euclidean distance of the final predicted and ground

truth positions at the final time step. Either are evaluated separately at the short time horizons 0.6s, 1.0s and 2.0s which also follows common practices of reporting trajectory predictions (Hudnell et al., 2019; Schuetz et al., 2024). As trajectory windows from the same pedestrian track may overlap and thus are not fully independent the evaluation units are aggregated at a track level before doing statistical testing, this reduces the dependence from overlapping prediction windows.

$$\text{mean_ade_until_horizon} = \frac{1}{m} \sum_{j=1}^m ADE_j,$$

$$\text{mean_fde_at_horizon} = \frac{1}{m} \sum_{j=1}^m FDE_j.$$

Runtime measures such as latency and throughput are recorded to verify whether the real-time constraints are met.

RQ1: Comparative performance under real-time constraints

After training and evaluating the Spatio-Temporal Transformer and the LSTM baseline on the same datasets, making the comparison paired, the difference between the baseline error and the Spatio-Temporal Transformer error is computed for each metric and horizon $d_i = e_{i,baseline} - e_{i,ST}$. Hence, a positive difference indicates an improvement of the Spatio-Temporal Transformer. This is followed by a paired sign-flip permutation test to test whether the mean paired difference is significantly greater than zero (Good, 2005). This permutation test is non-parametric as it does not require a normality assumption for the trajectory error differences. The effect size is quantified by a paired bootstrap confidence interval for the mean paired difference. Since the analysis is repeated across multiple prediction horizon for both ADE and FDE, Holm correction is applied to control the family-wise error rate (Holm, 1979). Runtime measures are also reported to verify the real-time budget

RQ2: Equivalence testing margin and TOST procedure

In order to identify the fraction of synthetic data that allows for practical equivalence with the real-world-trained model, a synthetic equivalence margin is determined in relation to the real-world-trained model performance.:

$$\delta = 0.05 \cdot \bar{e}_{\text{real}},$$

where $\bar{e}_{\text{real}}$ denotes the mean error of the real-trained model for the corresponding metric, prediction horizon, synthetic model or fraction, and aggregation level. Therefore, the synthetic-trained model is considered practically equivalent if its mean error differs from the real-trained model by no more than 5% of the real-trained model’s mean error for the specific ADE or FDE horizon being tested.

The paired difference is defined as

$$d_i = e_{i,\text{synthetic}} - e_{i,\text{real}},$$

where both errors are computed on the same evaluation unit. The implemented test is a paired Two One-Sided Tests (TOST) procedure using paired t-tests (Schuirmann, 1987). Let

$$\bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad SE = \frac{s_d}{\sqrt{n}}, \quad df = n - 1,$$

where s_d is the standard deviation of the paired differences. The lower one-sided test checks whether the synthetic-trained model is not more than δ better than the real-trained model,

$$H_{0,L} : \mu_d \leq -\delta \quad \text{against} \quad H_{1,L} : \mu_d > -\delta,$$

while the upper one-sided test checks whether the synthetic-trained model is not more than δ worse than the real-trained model,

$$H_{0,U} : \mu_d \geq \delta \quad \text{against} \quad H_{1,U} : \mu_d < \delta.$$

Equivalence is concluded only if both one-sided tests are significant at $\alpha = 0.05$. Equivalently, the 90% paired t -confidence interval for μ_d must lie entirely inside the equivalence interval

$$[-\delta, \delta].$$

Multiple equivalence tests are performed across metrics, horizons, and synthetic data fractions as described in the experiments. As none of the unadjusted RQ2 equivalence tests reached approximate equivalence, applying Holm correction would not change the conclusion (Holm, 1979). The unadjusted TOST results are therefore reported and would like to note that multiple testing correction would only make concluding on approximate equivalence harder.

RQ3: Cross-domain generalization

Cross domain generalisation performance is evaluated using a 2×2 domain design, incorporating the training and the test domain, which each has two variations: The real and synthetic data.

$$(\text{real}, \text{real}), \quad (\text{real}, \text{synthetic}), \quad (\text{synthetic}, \text{real}), \quad (\text{synthetic}, \text{synthetic}).$$

The analysis is performed separately for each metric and prediction horizon, using ADE and FDE aggregated at track level. Mixed-domain models may be summarized descriptively, but they are not included in the primary 2×2 factorial test.

For each metric horizon combination, the observed train-domain main effect, test-domain main effect, and train domain by test-domain interaction effect are computed. The train-domain main effect measures the absolute difference between real world data trained and synthetic data trained models, averaged over test domains. The test domain main effect measures the absolute difference between real and synthetic test performance, averaged over training domains. The interaction effect is computed as a difference in differences:

$$|(\bar{e}_{\text{real}, \text{real}} - \bar{e}_{\text{real}, \text{synthetic}}) - (\bar{e}_{\text{synthetic}, \text{real}} - \bar{e}_{\text{synthetic}, \text{synthetic}})|.$$

A large interaction effect indicates that the difference between real and synthetic test performance depends on the training domain.

To obtain p-values, the joint domain labels train domain, test domain are permuted across rows (Good, 2005). For each permutation, the same effect statistics are recomputed. The p-value is then estimated as the proportion of permutations whose effect statistic is at least as large as the observed effect:

$$p = \frac{c_{\text{extreme}} + 1}{n_{\text{perm}} + 1}.$$

Where c_{extreme} describes the number of random permutation that produce an effect at least as large as the observed effect. The number of permutations is $n_{\text{perm}} = 10,000$ to ensure a stable p value. Since the test is permutation-based, it does not assume normally distributed errors.

3.2 Quantitative Results

Main writer: Néo Deward

Reviewer: Nick Grebe

3.2.1 RQ1: Spatio-Temporal Transformer and LSTM

As hypothesised, the Spatio-Temporal Transformer outperforms the LSTM model when trained on the same dataset with a comparable parameter count. Figure 8 illustrates this advantage, showing a lower mean track-level error for the Transformer architecture.

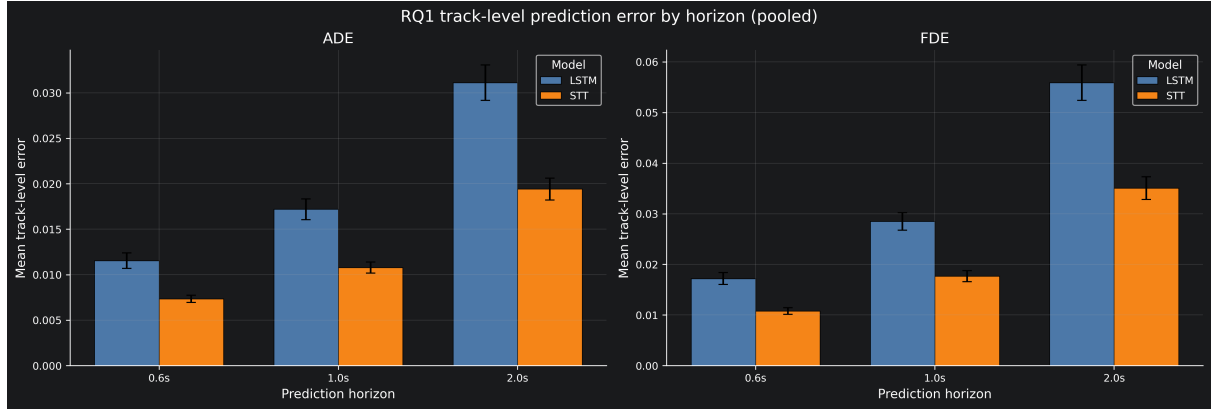


Figure 8: RQ1 Mean track-level error for the LSTM and Spatio-Temporal Transformer models.

Bootstrapped confidence intervals confirm that this performance advantage is statistically significant across the prediction horizon.

3.2.2 RQ2: Real vs. Synthetic Training

Figure 9 illustrates the relative performance gap between models trained on synthetic versus real data. The synthetic-trained model yields higher errors on real-world test data. Notably, this relative error gap plateaus regardless of the volume of synthetic training data provided.

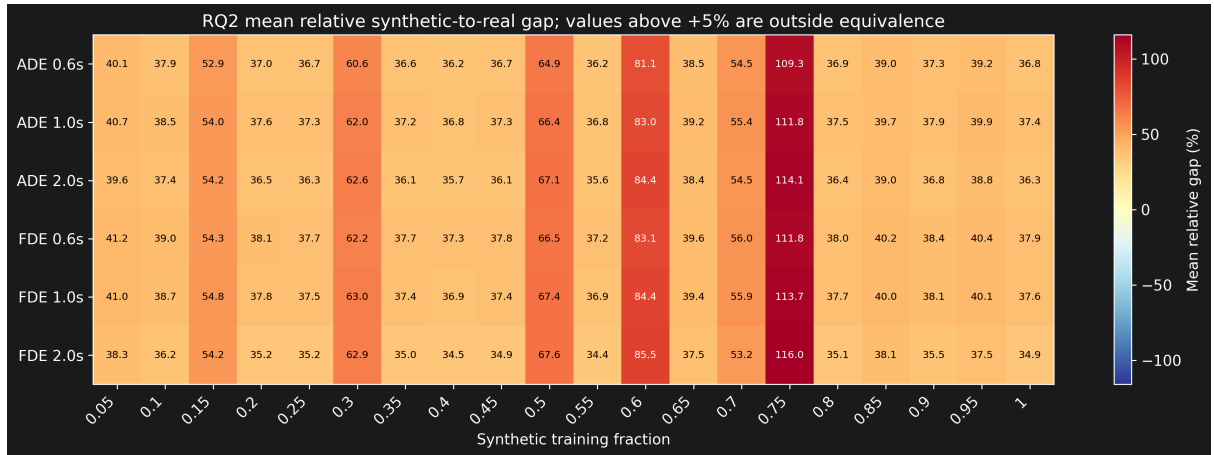


Figure 9: RQ2 mean relative synthetic to real gap

Figure 10 provides a granular view, confirming that the performance of the synthetic-trained model remains strictly outside the $\pm 5.0\%$ equivalence margin $[-\delta, \delta]$ across all individual MOT20

test splits.

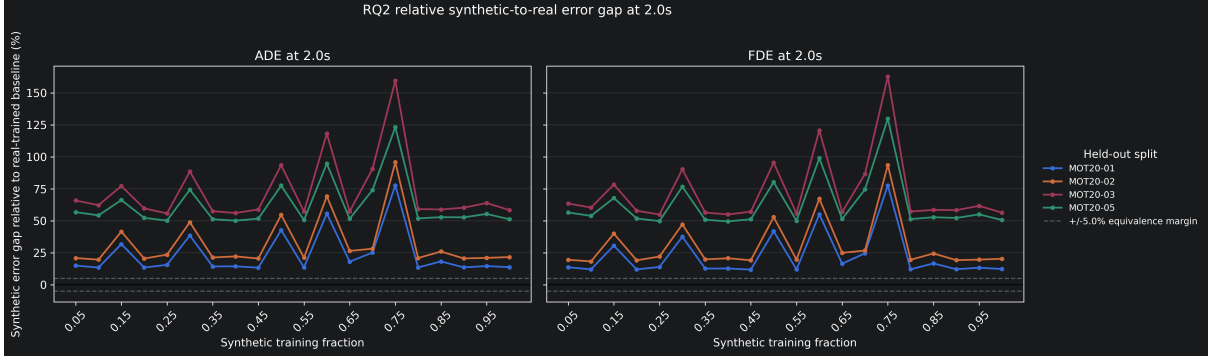
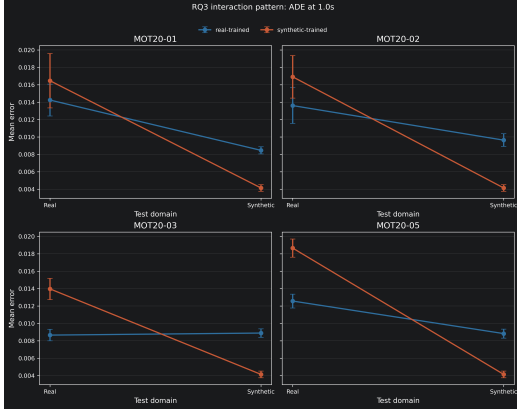


Figure 10: Relative synthetic to real error gap at the 2.0s prediction horizon

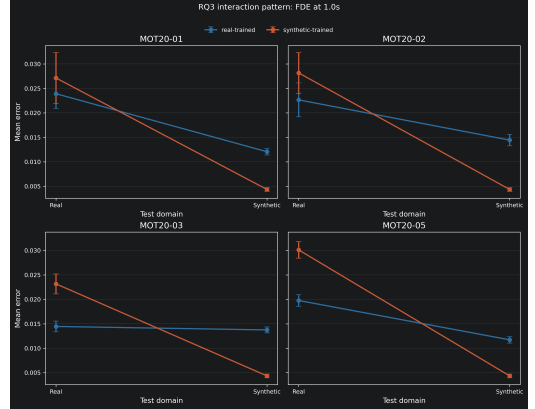
As observed in Figure 10, there is a pronounced “sawtooth” pattern indicating instability in the out-of-domain error during sequential training. The causes and implications of this phenomenon including catastrophic forgetting and game-engine artifacts are analysed in Section 4.1.

3.2.3 RQ3: Cross-Domain Generalisation

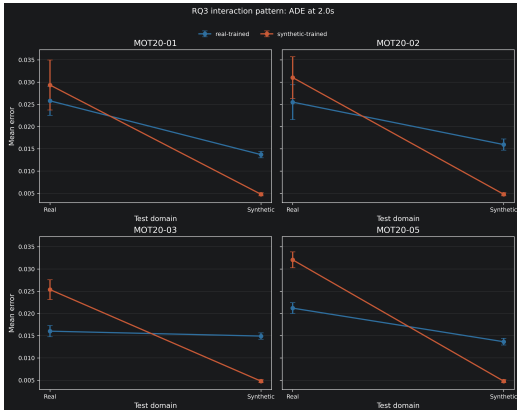
Figure 11 displays the mean errors broken down by the individual MOT20 test splits to evaluate cross-domain generalisation.



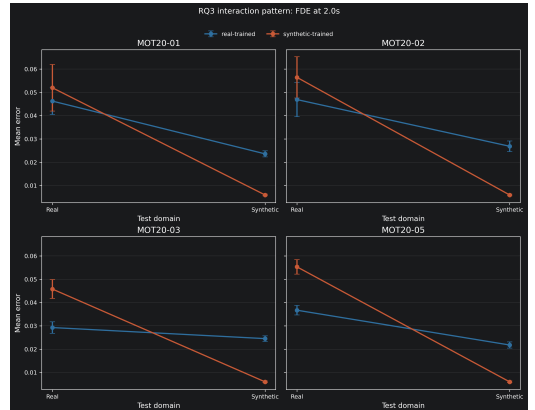
(a) ADE at 1.0s



(b) FDE at 1.0s



(c) ADE at 2.0s



(d) FDE at 2.0s

Figure 11: RQ3 cross-domain interaction patterns for ADE and FDE.

These subfigures reveal a consistent trend of asymmetric cross-domain generalisation. The model trained on real data achieves comparatively stable and robust results in both real-world and synthetic evaluation domains. In contrast, the synthetic-trained model degrades in performance when tested on real-world data, despite performing exceptionally well within its own domain. This shows that real-world training confers generalised robustness and synthetic training alone creates specialised models that fail to generalise.

4 Discussion

4.1 Interpretation of Findings

Main writer: Noah Nuelandt

Reviewer: Nick Grebe, Keez Cuijpers

Contributors: Claire Bams, Néo Deward

The first research question shows that the Spatio-Temporal Transformer benefits from combining temporal modelling with spatial and pedestrian interaction information. Compared with the Social-LSTM baseline, the transformer architecture can attend to the observed trajectory more directly and also incorporates local and global crowd context. This implies that richer spatio-temporal representations are possibly useful for short term pedestrian trajectory prediction, especially in crowded scenes that are similar to the ones found in the MOT20 Dataset

The second research question investigated the viability of synthetic data for the task. None of the synthetic data fractions were found to be sufficient for the model to generalise well enough to be applicable to real world data. There can be a number of reasons for this, but the most prominent explanation is the homogeneity of standard synthetic data. A lack of diversity can easily cause overfitting and an inability to generalise to unseen and nonstandard data. This hypothesis is further reinforced by the high variance in validation losses, which hints that the model fails to improve on the test set despite a larger training set. One possible solution is to generate synthetic data that more closely resembles real-world conditions by introducing unconventional camera angles, a slight camera shake, and unpredictable pedestrian paths.

The third research question evaluated cross-domain generalisation by testing each model on both real-world and synthetic data. The real-world trained model appears to be more robust, providing consistent results in both domains. The synthetic model in contrast outperforms the real-world model on the synthetic test set, while performing worse on the real-world test set. This can be interpreted as overfitting within the synthetic model, as it fails to generalise, resulting in higher prediction error on out-of-domain data. Ultimately, Real-world data remains necessary for achieving optimal real-domain trajectory prediction in this setup.

Training Instability on Synthetic Data

As previously shown in Figure 10, the model illustrates instability in its out-of-domain error. Because the model trains sequentially on fractions of the MOTSynth dataset, it appears susceptible to anomalous batches (Toneva et al., 2018). Similar instability mechanism have been found in literature and may provide a possible explanation for the observed behaviour. Physics glitches inherent to game engines can trigger gradient shocks and temporarily destabilise the model’s weights (Pascanu et al., 2013). The error spikes observed (e.g., at the 0.75 training fraction) likely reflect catastrophic forgetting due to a distributional data shift. Given the large overall scale of the MOTSynth dataset, a 5% training increment still equates to gigabytes of sequential trajectory data. exposure to such a massive block of game-engine artefacts provides

enough gradient updates to restructure the model’s parameters. During these events, the model drastically overwrites its generalised kinematic knowledge to adapt to highly specific artefacts in the synthetic batch before recovering in further splits (Kirkpatrick et al., 2017; McCloskey & Cohen, 1989). This demonstrates “shortcut learning,” where the model exploits simulation loopholes to minimise training loss (Geirhos et al., 2020).

4.2 Limitations

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

The main limitation of this research project is the dependency on simulated data from one point in time. The advancements in simulated data are a continuous endeavour. The more compute resources become available for such simulations – as they have over the last couple of years – the likelier they’ll become a approximation of the real world. Additionally, the investigation focuses on one source of synthetic data, which may carry undiscovered biases introduced by its creators. Nevertheless, the design choices of the experiments and models are replicable to other sources of synthetic data which remain to be tested.

An inherent issue also stems from the irregularity of real world pedestrian behaviour, as trying to predict such behaviour would classify as anomaly detection instead. This was labelled as being outside of the scope of this research project, but may nonetheless be highly relevant to future work on such experiments.

Furthermore this research was strongly limited in available compute and storage, which led to the analysed data and its differences being relatively small. The MOT20 and MOTSynth dataset do not reflect all possible combinations of pedestrian interactions or layouts. The approach serves as a mere approximation of such, and the findings cannot be claimed to generalise for all layouts and circumstances. A stronger conclusion would require vastly larger compute and access to data which was not feasible within the scope of this project.

4.3 Future Work

Main writer: Nick Grebe

Reviewer: Noah Nuelandt

Future work should prioritise expanding compute resources to evaluate on larger and more diverse datasets enabling stronger conclusions about model behaviour under real versus synthetic training conditions. Additionally, further investigation in the simulation process of the artificial data, identifying which properties, such as density, occlusion, turning behaviour and viewpoint variation are most suited for synthetic to real world model transfer. Furthermore, an investigation of domain adaptation would be highly relevant, analysing the effects of fine-tuning the fully synthetic models on small real world datasets, both evaluating on clean data and its robustness to noisy tracking which is a realistic challenge in low-latency real-world applications.

5 Conclusion

Main writer: Nick Grebe

Reviewer: Claire Bams

This project investigated whether synthetic data can replace real-world data for pedestrian trajectory prediction.

Overall, the results provide three main findings. Firstly, the Spatio-Temporal transformer achieved lower ADE and FDE than the Social-LSTM baseline. This suggests that modelling temporal dependencies together with spatial and social attention improves trajectory prediction in crowded environments, such in the utilised MOT20-dataset. Secondly the synthetic data model did not achieve any practical equivalence with the real-world-trained model, none of the analysed training fractions entered the 5% equivalence margin. Hence leading to the conclusion that increasing the amount of synthetic data did not close the cross real-to-synthetic domain performance gap. Lastly the cross domain evaluation showed an asymmetric pattern: The real-world-trained model transferred more robustly across domains, where on the other hand the synthetic-trained model did not perform well on the real world data, but decently on its own domain.

Answering the title of this research, we cannot say that synthetic data can replace real-world data for pedestrian trajectory prediction, where the main gap appears to be the synthetic-to-real domain and not the sheer amount of data. We would nonetheless like to stress that these findings do not mean that synthetic data is useless, nor does it prove that synthetic data can and will never be able to produce models that are equivalent to their real-world counterparts. This leads to promising next steps as one would be posing the question if synthetic pretraining, data augmentation or controlled scenario generation would in fact improve accuracy.

A Supplementary Visualisations and Performance Metrics

Main writer: Néo Deward

Reviewer: Nick Grebe

This appendix provides performance data regarding the architectural advantages of the Spatio-Temporal Transformer (STT) over the Social-LSTM across real-world MOT20 test sequences.

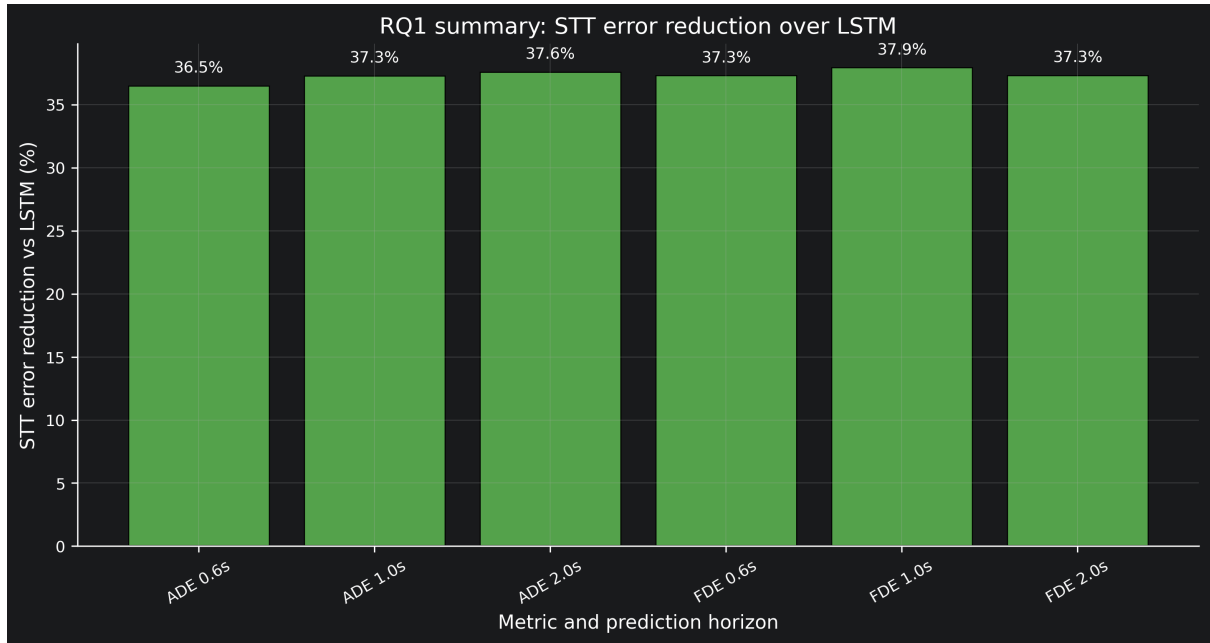


Figure 12: Aggregated relative error reduction of the STT compared to the LSTM. The STT achieves a consistent performance gain of approximately 36–38% across all ADE and FDE prediction horizons.

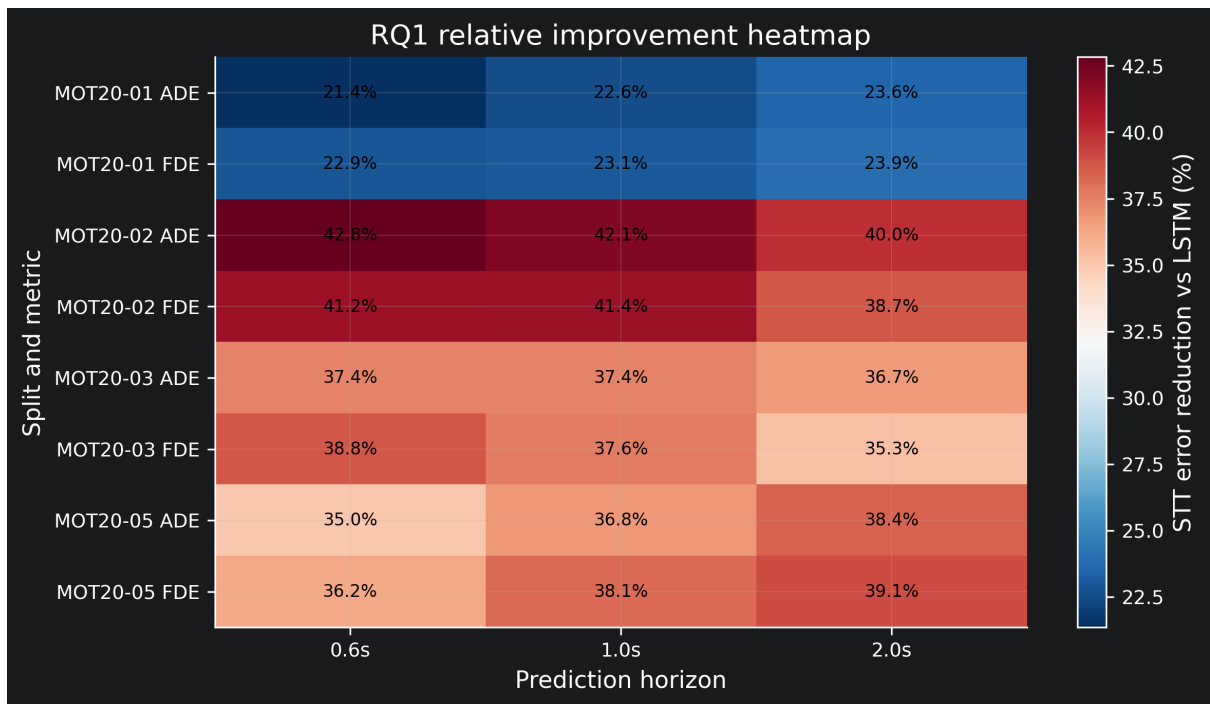


Figure 13: Heatmap detailing the relative improvement (%) of the STT over the LSTM baseline, broken down by individual MOT20 sequence splits.

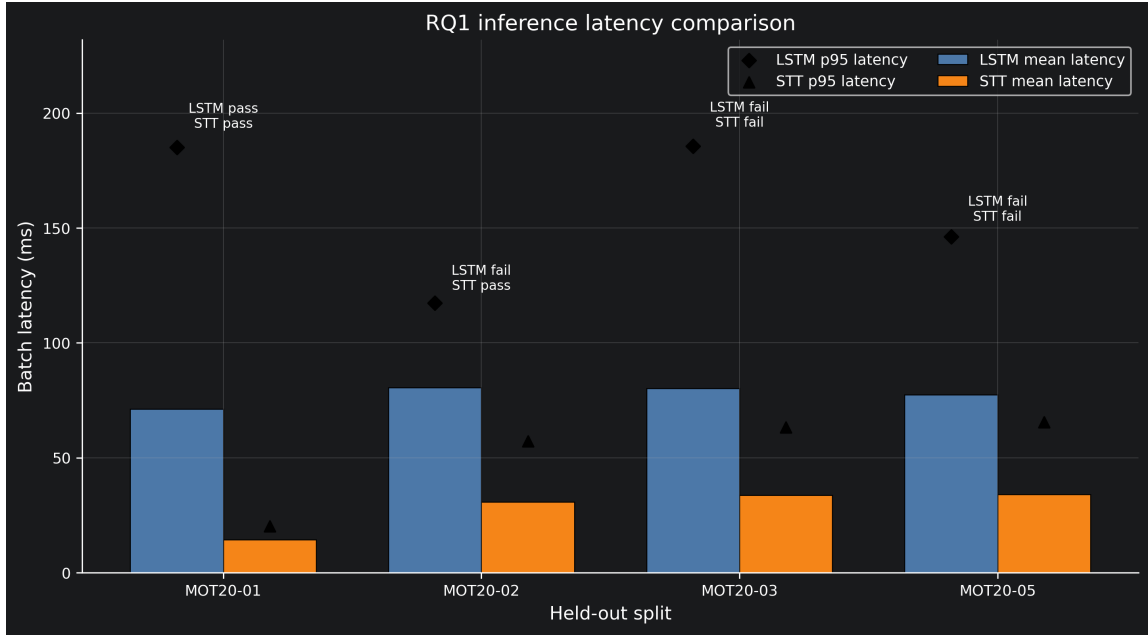


Figure 14: Comparison of batch inference latency (ms). The STT maintains significantly lower mean and 95th-percentile latency than the LSTM, ensuring real-time viability. While the LSTM frequently exceeds the 100ms threshold in complex scenes, the STT remains consistently efficient.

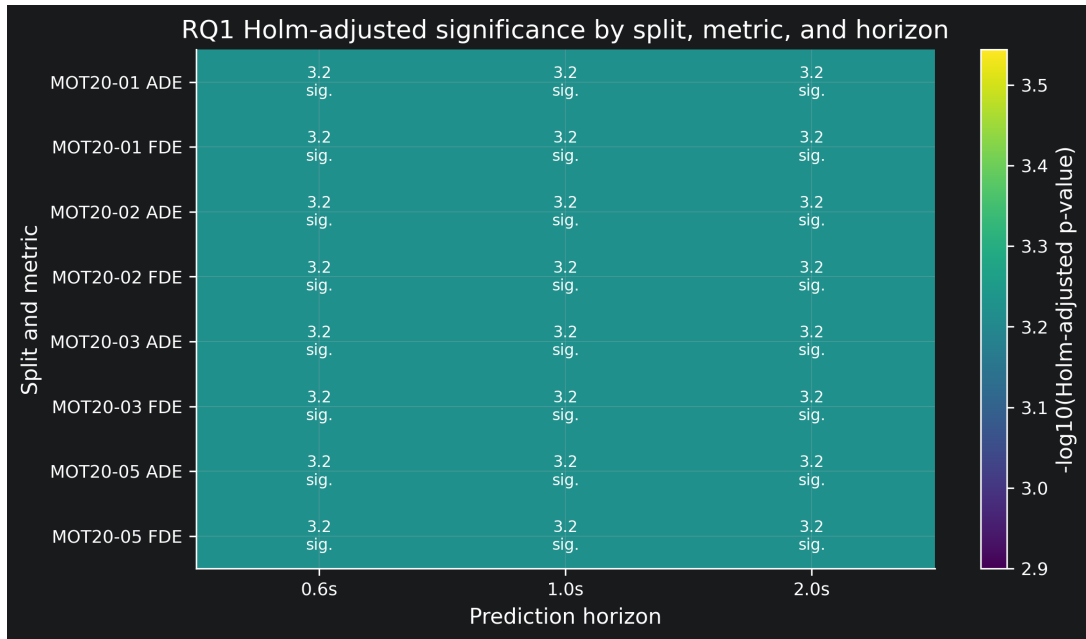


Figure 15: Statistical significance matrix reporting Holm-adjusted p-values on a negative log scale ($-\log_{10}(p)$). Higher values represent stronger statistical evidence.

References

Alahi, A., Goel, K., Ramanathan, V., Robicquet, A., Li, F.-F., & Savarese, S. (2016). Social LSTM: Human Trajectory Prediction in Crowded Spaces. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 961–971. <https://doi.org/10.1109/CVPR.2016.110>

- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer Normalization. *arXiv preprint arXiv:1607.06450*. <https://doi.org/10.48550/arXiv.1607.06450>
- Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., & Leal-Taixé, L. (2021). MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. *International Journal of Computer Vision*, 129(4), 845–881. <https://doi.org/10.1007/s11263-020-01393-0>
- European Data Protection Board. (2020, January). Guidelines 3/2019 on processing of personal data through video devices.
- Eurostat. (2026). *Road safety statistics in the eu*. European Commission. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Road_safety_statistics_in_the_EU
- Fabbri, M., Braso, G., Maugeri, G., Cetintas, O., Gasparini, R., Osep, A., Calderara, S., Leal-Taixe, L., & Cucchiara, R. (2021). Motsynth: How can synthetic data help pedestrian detection and tracking? *arXiv*. <https://doi.org/10.48550/arxiv.2108.09518>
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11), 665–673. <https://doi.org/10.1038/s42256-020-00257-z>
- Good, P. I. (2005). *Permutation, parametric, and bootstrap tests of hypotheses* (3rd ed.). Springer.
- Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*. <https://doi.org/10.48550/arXiv.1606.08415>
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- Huang, Y., Bi, H., Li, Z., Mao, T., & Wang, Z. (2019). STGAT: Modeling Spatial-Temporal Interactions for Human Trajectory Prediction. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 6272–6281. <https://doi.org/10.1109/ICCV.2019.00637>
- Hudnell, M., Karnchanachari, N., Qiao, Y., Lu, W., Chen, X., & Urtasun, R. (2019). Robust Aleatoric Modeling for Future Vehicle Localization. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. https://openaccess.thecvf.com/content_CVPRW_2019/html/Precognition/Hudnell_Robust_Aleatoric_Modeling_for_Future_Vehicle_Localization_CVPRW_2019_paper.html
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526. <https://doi.org/10.1073/pnas.1611835114>
- Liang, J., Jiang, L., & Hauptmann, A. (2020). SimAug: Learning Robust Representations from Simulation for Trajectory Prediction. *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIII*, 12358, 275–292. https://doi.org/10.1007/978-3-030-58601-0_17
- McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of learning and motivation*, 24, 109–165.
- Mohamed, A., Qian, K., Elhoseiny, M., & Claudel, C. (2020). Social-STGCNN: A Social Spatio-Temporal Graph Convolutional Neural Network for Human Trajectory Prediction. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14424–14432. <https://doi.org/10.1109/CVPR42600.2020.01443>
- Mohamed, A., Zhu, D., Vu, W., Elhoseiny, M., & Claudel, C. (2022). Social-Implicit: Rethinking Trajectory Prediction Evaluation and the Effectiveness of Implicit Maximum Likelihood Estimation. *Computer Vision ECCV 2022*. https://doi.org/10.1007/978-3-031-20047-2_27
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *International conference on machine learning*, 1310–1318.

- Salzmann, T., Ivanovic, B., Chakravarty, P., & Pavone, M. (2020). Trajectron++: Dynamically-Feasible Trajectory Forecasting With Heterogeneous Data. *Computer Vision – ECCV 2020*, 12363, 683–700. https://doi.org/10.1007/978-3-030-58523-5_40
- Schöller, C., Aravantinos, V., Lay, F., & Knoll, A. (2020). What the Constant Velocity Model Can Teach Us About Pedestrian Motion Prediction. *IEEE Robotics and Automation Letters*, 5(2), 1696–1703. <https://doi.org/10.1109/LRA.2020.2969925>
- Schuetz, E., Rehder, E., & Stiller, C. (2024). A Review of Trajectory Prediction Methods for the Vulnerable Road User. *Robotics*, 13(1), 1. <https://doi.org/10.3390/robotics13010001>
- Schuurmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6), 657–680. <https://doi.org/10.1007/BF01068419>
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958. Retrieved June 4, 2026, from <https://jmlr.org/papers/v15/srivastava14a.html>
- Toneva, M., Sordoni, A., des Combes, R. T., Trischler, A., Bengio, Y., & Gordon, G. J. (2018). An empirical study of example forgetting during deep neural network learning. *International Conference on Learning Representations*.
- Uhlemann, N., Fent, F., & Lienkamp, M. (2024). Evaluating Pedestrian Trajectory Prediction Methods With Respect to Autonomous Driving [Preprint version available as arXiv:2308.05194]. *IEEE Transactions on Intelligent Transportation Systems*. <https://arxiv.org/abs/2308.05194>
- Wilson, B., Qi, W., Agarwal, T., Lambert, J., Singh, J., Khandelwal, S., Pan, B., Kumar, R., Hartnett, A., Pontes, J. K., Ramanan, D., Carr, P., & Hays, J. (2023). Argoverse 2: Next generation datasets for self-driving perception and forecasting. *arXiv preprint arXiv:2301.00493*. <https://arxiv.org/abs/2301.00493>
- Yu, C., Ma, X., Ren, J., Zhao, H., & Yi, S. (2020). Spatio-Temporal Graph Transformer Networks for Pedestrian Trajectory Prediction. *Computer Vision – ECCV 2020*, 12357, 507–523. https://doi.org/10.1007/978-3-030-58610-2_30
- Yuan, Y., Weng, X., Ou, Y., & Kitani, K. M. (2021). AgentFormer: Agent-Aware Transformers for Socio-Temporal Multi-Agent Forecasting. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 9813–9823. <https://doi.org/10.1109/ICCV48922.2021.00967>